

Task 1 — Money Detective

Date	Description	Amount (\$)
2026-06-01	Salary	+2500
2026-06-02	Walmart	-85
2026-06-03	Netflix	-15.99
2026-06-04	Uber	-18
2026-06-05	Spotify	-9.99
2026-06-06	McDonald's	-12
2026-06-07	Amazon	-120
2026-06-08	Gym Membership	-40
2026-06-09	Netflix	-15.99
2026-06-10	Uber	-18
2026-06-11	Electricity Bill	-75
2026-06-12	Grocery Store	-92
2026-06-13	Amazon	-120

Here's a simple Python script that analyzes the transactions and finds:

1. **Recurring charges** (same merchant appears multiple times)
2. **Forgotten subscriptions** (small recurring charges that look like subscriptions)
3. **Duplicate payments** (same merchant with the same amount more than once)

```
import pandas as pd
```

```
# -----
```

```
# Sample Data
```

```
# -----
```

```
data = {
```

```
    "Date": [
```

```
        "2026-06-01", "2026-06-02", "2026-06-03", "2026-06-04",
```

```
        "2026-06-05", "2026-06-06", "2026-06-07", "2026-06-08",
```

```
        "2026-06-09", "2026-06-10", "2026-06-11", "2026-06-12",
```

```
        "2026-06-13"
```

```

],
"Description": [
    "Salary", "Walmart", "Netflix", "Uber", "Spotify",
    "McDonald's", "Amazon", "Gym Membership",
    "Netflix", "Uber", "Electricity Bill",
    "Grocery Store", "Amazon"
],
"Amount": [
    2500, -85, -15.99, -18, -9.99, -12, -120, -40,
    -15.99, -18, -75, -92, -120
]
}

```

```

df = pd.DataFrame(data)
df["Date"] = pd.to_datetime(df["Date"])

```

```

# -----
# 1. Find Recurring Charges
# -----
print("="*50)
print("Recurring Charges")
print("="*50)

```

```

recurring = (
    df[df["Amount"] < 0]
    .groupby("Description")
    .agg(
        Count=("Description", "count"),
        Amount=("Amount", "first")
    )
    .query("Count > 1")
)

```

```

if recurring.empty:
    print("No recurring charges found.")
else:
    print(recurring)

```

```

# -----
# 2. Forgotten Subscriptions
# Small recurring payments (< $50)
# -----
print("\n" + "="*50)
print("Possible Forgotten Subscriptions")

```

```

print("="*50)

subscriptions = recurring[
    recurring["Amount"].abs() <= 50
]

if subscriptions.empty:
    print("No possible subscriptions found.")
else:
    print(subscriptions)

# -----
# 3. Duplicate Payments
# Same merchant + same amount
# -----
print("\n" + "="*50)
print("Duplicate Payments")
print("="*50)

duplicates = df[
    df.duplicated(
        subset=["Description", "Amount"],
        keep=False
    )
].sort_values(["Description", "Date"])

if duplicates.empty:
    print("No duplicate payments found.")
else:
    print(duplicates)

# -----
# Summary
# -----
print("\n" + "="*50)
print("Summary")
print("="*50)

print(f'Recurring Merchants: {len(recurring)}')
print(f'Possible Subscriptions: {len(subscriptions)}')
print(f'Duplicate Transactions: {len(duplicates)}')

```

Expected Output

```
=====
```

Recurring Charges

```
=====
      Count Amount
Description
Amazon      2 -120.00
Netflix     2  -15.99
Uber        2  -18.00
```

Possible Forgotten Subscriptions

```
=====
      Count Amount
Description
Netflix     2  -15.99
Uber        2  -18.00
```

Duplicate Payments

```
=====
      Date Description Amount
2026-06-07   Amazon -120.00
2026-06-13   Amazon -120.00
2026-06-03   Netflix -15.99
2026-06-09   Netflix -15.99
2026-06-04    Uber -18.00
2026-06-10    Uber -18.00
```

Summary

```
=====
Recurring Merchants: 3
Possible Subscriptions: 2
Duplicate Transactions: 6
```

Improvements for real bank statements

For production use, you could make the script smarter by:

- Detecting monthly billing patterns (28–31 days apart) instead of just repeated merchants.
- Using fuzzy matching so "NETFLIX.COM" and "Netflix" are treated as the same merchant.
- Flagging subscriptions based on merchant categories (Netflix, Spotify, Adobe, etc.).
- Distinguishing true duplicate payments (same day or within a few days) from normal monthly recurring charges.

- Generating a report (CSV, Excel, or PDF) with estimated monthly subscription costs and annual spending.

how this code working understand me in simple english

Let's understand it as if you're a beginner. Imagine you're a bank employee checking a customer's transactions.

Step 1: Import Pandas

import pandas as pd

What is happening?

We are importing the **Pandas** library.

Think of Pandas as **Microsoft Excel inside Python**.

It helps us work with tables.

Step 2: Create the data

```
data = {  
    "Date": [...],  
    "Description": [...],  
    "Amount": [...]  
}
```

Here we are creating a dictionary.

It looks like this:

Date	Description	Amount
2026-06-01	Salary	2500
2026-06-02	Walmart	-85

This is just the raw information.

Step 3: Convert into a DataFrame

```
df = pd.DataFrame(data)
```

This converts the dictionary into a table.

Now Python sees this:

Date	Description	Amount
2026-06-01	Salary	2500
2026-06-02	Walmart	-85
2026-06-03	Netflix	-15.99

Think of `df` as an Excel sheet.

Step 4: Convert Date

```
df["Date"] = pd.to_datetime(df["Date"])
```

Before this line

2026-06-01

is just text.

After this line

2026-06-01

becomes a real date.

Now Python can calculate things like

- days

- months
 - years
-

Step 5: Ignore Salary

```
df[df["Amount"] < 0]
```

Let's understand this slowly.

Imagine this table

Description	Amount
Salary	2500
Netflix	-15.99
Uber	-18
Amazon	-120

The condition

`Amount < 0`

means

Show me only money that went **out**.

Result

Description	Amount
Netflix	-15.99
Uber	-18
Amazon	-120

Salary disappears because it is positive.

Step 6: Group by Description

```
.groupby("Description")
```

Python groups the same company together.

Before grouping

Netflix
Uber
Amazon
Netflix
Amazon
Uber

After grouping

Netflix
-15.99
-15.99

Uber
-18
-18

Amazon
-120
-120

Think of making separate folders for each company.

Step 7: Count

```
Count=("Description","count")
```

Python asks

How many times did this company appear?

Example

Netflix

Netflix
Netflix

Count = 2

Amazon

Amazon
Amazon

Count = 2

Spotify

Spotify

Count = 1

Step 8: First Amount

Amount=("Amount","first")

Python takes the first payment amount.

Example

Netflix

-15.99
-15.99

First amount

-15.99

Amazon

-120
-120

First amount

-120

Step 9: Keep only repeated companies

```
.query("Count > 1")
```

This means

Only keep merchants that appear more than once.

Before

Company	Count
Netflix	2
Uber	2
Amazon	2
Spotify	1
Walmart	1

After

Company	Count
Netflix	2
Uber	2
Amazon	2

Spotify disappears.

Step 10: Possible subscriptions

```
subscriptions = recurring[  
    recurring["Amount"].abs() <= 50  
]
```

This means

Take recurring payments that are **\$50 or less**.

Example

Merchant	Amount
Netflix	-15.99
Spotify	-9.99
Uber	-18
Amazon	-120

Python changes the negative numbers to positive for comparison:

Merchant	Absolute Value
Netflix	15.99
Spotify	9.99
Uber	18
Amazon	120

Then it keeps only values less than or equal to 50.

Result

Netflix
Spotify
Uber

Amazon is removed because $120 > 50$.

Step 11: Duplicate Payments

```
duplicates = df[
    df.duplicated(
        subset=["Description", "Amount"],
        keep=False
    )
]
```

This asks

Did the same merchant charge the same amount more than once?

Example

Merchant	Amount
----------	--------

Netflix	-15.99
---------	--------

Netflix	-15.99
---------	--------

Duplicate 

Another example

Merchant	Amount
----------	--------

Amazon	-120
--------	------

Amazon	-120
--------	------

Duplicate 

Another example

Merchant	Amount
----------	--------

Spotify	-9.99
---------	-------

Only one payment

Not duplicate 

Step 12: Print Results

`print(recurring)`

Shows recurring merchants.

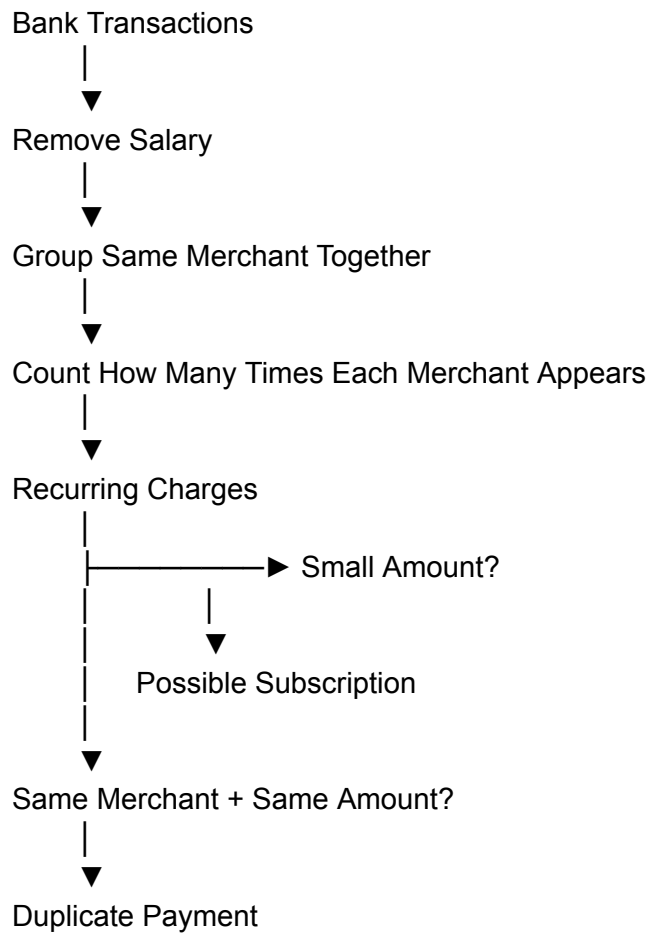
`print(subscriptions)`

Shows possible subscriptions.

```
print(duplicates)
```

Shows duplicate payments.

Overall Flow



The key idea is that **Pandas lets you ask questions about your table**. Each line of code filters, groups, or counts the data to answer a specific question, much like using filters and formulas in Excel.